ECE641 Project 3 Report

Tannyr Singleterry

Dr. Don Gruenbacher

4/10/2026

Scope of Project

The scope of this project was to take the SDRAM built in Project 2 and update the SDRAM memory controller to allow page burst writes and reads of a maximum of 512 words. Second, we added the ability to read an image from the SDRAM and output the image to the HDMI interface.

There was a provided updated SDRAM controller (sdram_ctrl_v3.v) to replace the one used in PR2. This updated version changes the burst length from 8 to a page burst mode where reads and writes continue on every clock cycle until a burst terminate command is executed. There was also a change to perform a precharge before returning to the Idle state. The auto refresh was also modified to perform 6 refresh cycles each time the start_refresh input is asserted, with the horizontal sync signal from the HDMI system connected to start_refresh. Priority capture logic was also added so that if a read, write, or refresh are submitted at the same time, they are all captured and executed in order of priority: refresh first, then write, then read.

Other changes made during this project included adjusting the SDRAM PLL to run at 148.5 MHz, adjusting both the address and data FIFOs to support two clocks — one for the read port and one for the write port. Each data FIFO was also made 1024 words long to support two full 512 word page bursts. The data FIFOs also expose a used w vector for the number of words currently stored, which is used instead of the partial full and partial empty flags.

Design Strategy

The design builds on the Project 2 hierarchy by starting from the Terasic HDMI_out_RTL demonstration project and adding the PR2 modules into it. The full module hierarchy is: golden_top.v, uart_tx.v, uart_rx.v, uart_mgr.v, sdram_mgr.v, sdram_ctrl_v3.v, fifo_24.v, fifo_32.v, sevseg_dec.v, along with the Terasic provided vpg.v, VEDIO_PLL, AUDIO_PLL IP cores among other files. These were the ones that I looked at and changed.

The VEDIO_PLL was configured with three output clocks: outclk_0 at 148.5 MHz and 0 degree phase as the system clock for SDRAM, UART, and the HDMI FSM; outclk_1 at 148.5 MHz with 5000 ps phase shift to drive DRAM_CLK; and outclk_2 at 74.25 MHz as a reference for the

HDMI pixel clock domain. The vpg module was kept at the 1920x1080p60 timing configuration to match the connected monitor.

The FIFOs were redesigned as dual clock variants (fifo_24.v and fifo_32.v) using gray code pointer synchronization for safe clock domain crossing. The data FIFOs are 1024 words deep and expose an 11 bit used output on the write clock domain. The address FIFOs use both ports at 148.5 MHz since addresses only flow between the UART manager and SDRAM manager on the same clock domain.

The SDRAM manager FSM retained the same seven states from PR2 but was updated for 512 word page bursts. The Check_Write threshold was raised to 512 words in the write data FIFO before issuing a burst. The burst counter was widened to 10 bits to count up to 511. The start_refresh input on the v3 controller is connected to HDMI_TX_HS so that 6 refresh cycles fire on every horizontal sync pulse, satisfying the 8192 refresh per 64ms requirement without conflicting with long burst operations.

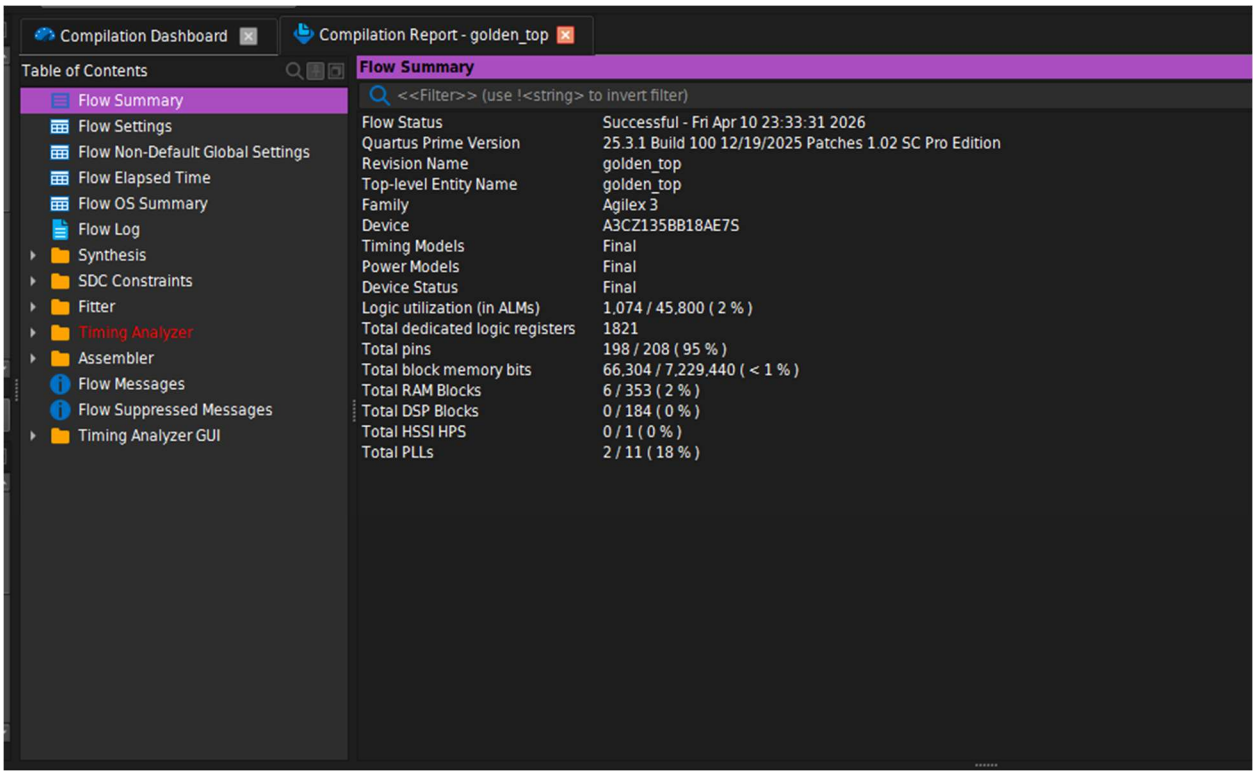
The uart_mgr was updated with several fixes from PR2. The edge detection for new_rx was corrected from XOR to AND to detect only rising edges and prevent the FSM from processing the same byte twice. The TX path was split into Prep and Fire states so that tx_word is stable for a full clock cycle before tx_start is asserted. A new bulk write command was added using lowercase 'w' (0x77) to handle the professor's RGB file format, which sends a 6 character address followed by 512 x 8 character pixel words per line. The counter was widened to 12 bits to handle the 4096 nibbles per bulk write command.

An HDMI read FSM was added inside sdram_mgr to handle Activity 2. This FSM runs at 148.5 MHz and has three states: HDMI_Idle, HDMI_Prefill, and HDMI_Stream. On each vsync rising edge the FSM resets the frame address to 0x000000 and begins pushing 512 word read address bursts into the read address FIFO. Once enough data has been prefetched, the FSM enters HDMI_Stream and outputs pixel data from the read data FIFO to the HDMI display during active pixel periods. An image_loaded flag in golden_top prevents the HDMI FSM from streaming until all 1800 bursts of the 720p image have been written to SDRAM, keeping the color bar on screen until the image is ready. I was running into issues with a image wanting to appear before it was done transfering

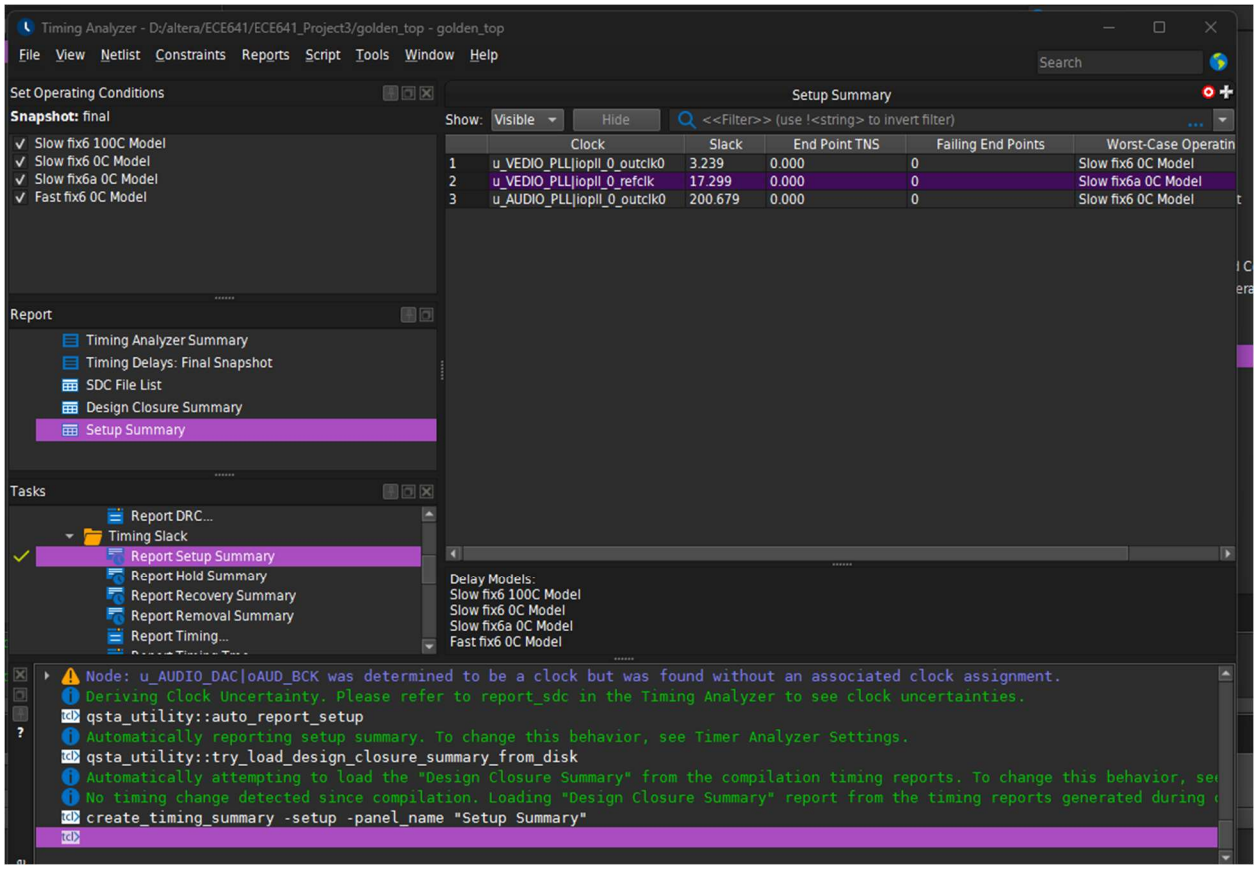
The pixel output uses img_pixel_cnt and img_line_cnt to track position within the 1280x720 image region inside the 1920x1080 frame. Pixels outside the image region fall back to the vpg color bar output through a mux on HDMI_TX_D. An img_line_active flag limits FIFO reads to the first 1280 pixels of each display line. A drain stage then silently discards 256 pixels per line to keep the FIFO aligned to 512 word burst boundaries, since 1280 is not a multiple of 512. My monitor I was using for testing was hard cap at 1920x1080

Results of Each Design

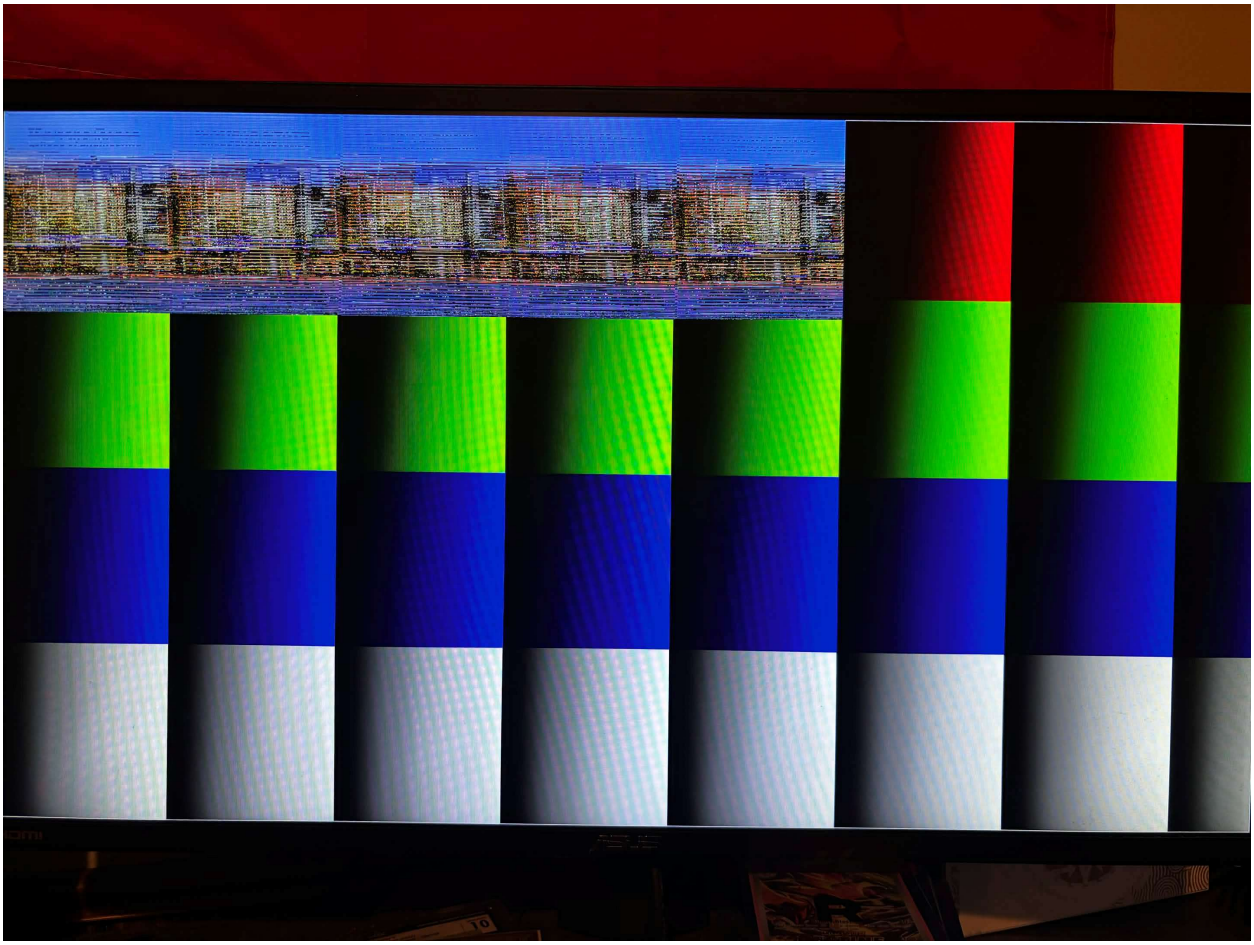
Below is the compilation report from Quartus:



Below is the timing report from Quartus:



Below is the image on my monitor from activity 2:



What I learned.

One of the biggest things I learned during this project was how much complexity comes from crossing clock domains. Moving from a single clock PR2 design to a multi clock PR3 design required careful thought about where each signal lives and how data moves between the 148.5 MHz and 74.25 MHz domains. The gray code FIFO synchronization was an important concept to understand and implement correctly. I learned this gray code technique mostly from this reddit thread trying to find solutions to this.

I also learned a lot about how HDMI timing works at the signal level. Connecting the HDMI FSM to vsync, hsync, and DE signals and using them to track pixel and line positions within a frame was something I had not done before. Understanding how the active display area relates to the full frame timing including blanking periods was essential for getting the image to appear in the correct location on screen.

The burst alignment issue was another important lesson. Because 1280 pixels per image line is not a multiple of 512, each burst crosses a row boundary. Without draining the leftover 256 pixels at the end of each line, the FIFO falls out of alignment and the image becomes scrambled after just two rows. This showed how important it is to think about memory access patterns at the burst level and not just the pixel level.

Conclusion

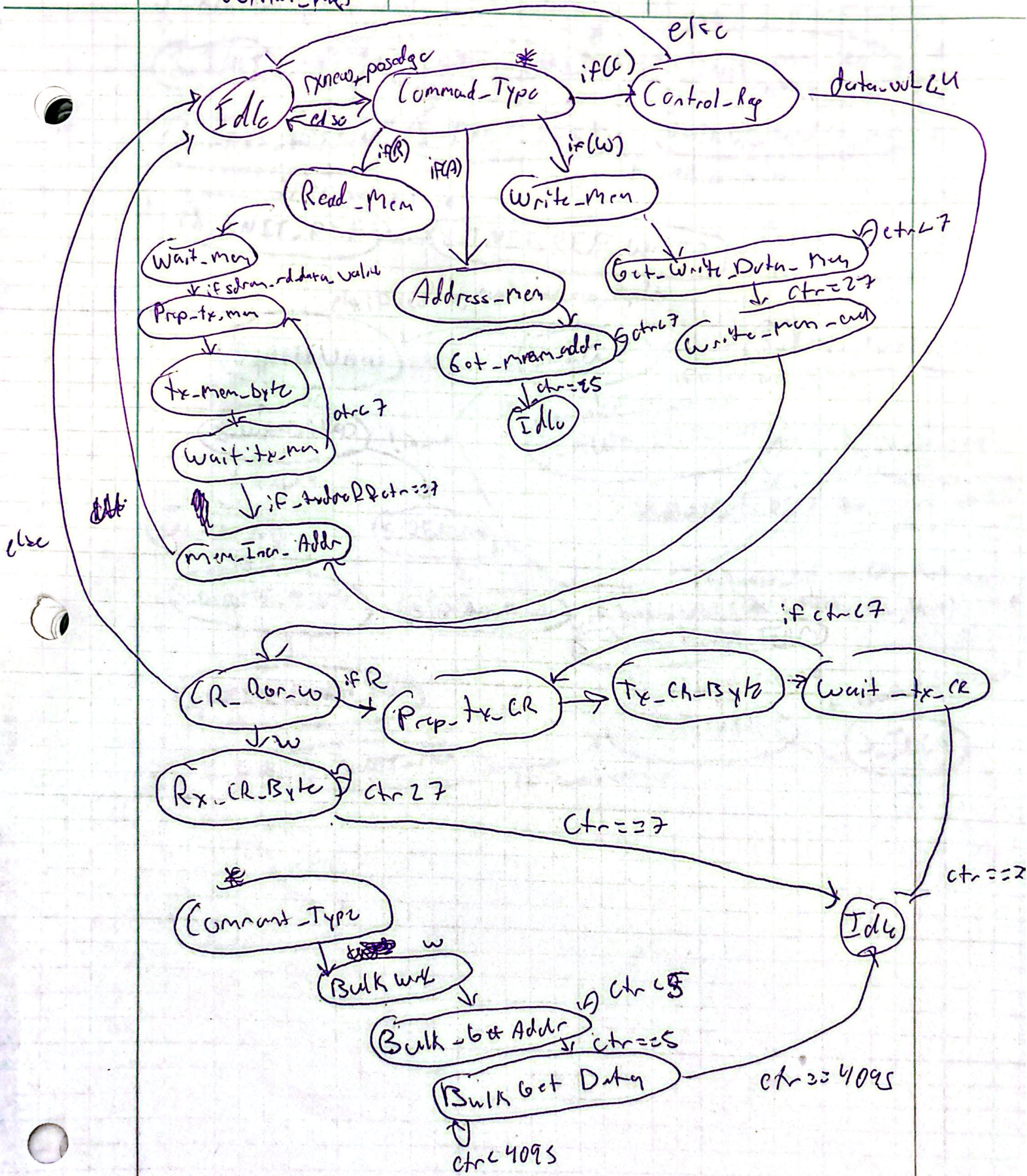
This project was significantly more complex than Project 2. Integrating the Terasic HDMI demonstration project with the SDRAM and UART system from PR2 required careful management of multiple clock domains, updated FIFOs, a new SDRAM controller, and a new HDMI streaming FSM. The UART, SDRAM write path, and HDMI color bar output all worked correctly. The image display was partially working with the College of Engineering Zoom background visible on screen, though the image alignment was still being worked on at the time of submission due to the 1280 pixel line width not being a clean multiple of the 512 word burst size.

Appendix

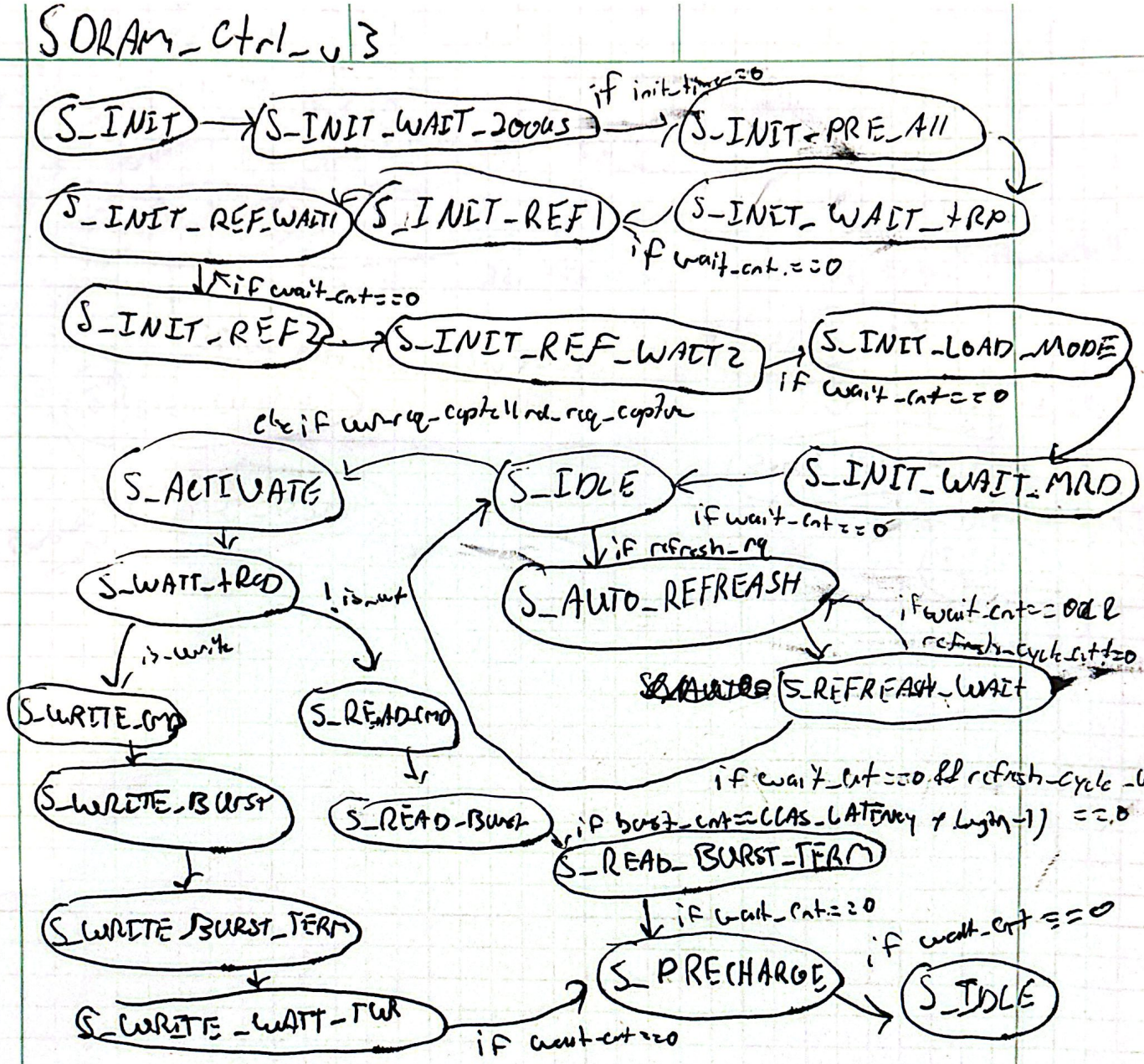
Here is the link to my git repo for this project: https://github.com/Tannyr-Singleterry/ECE641_Project3

State machine drawings are attached at the end of this report.

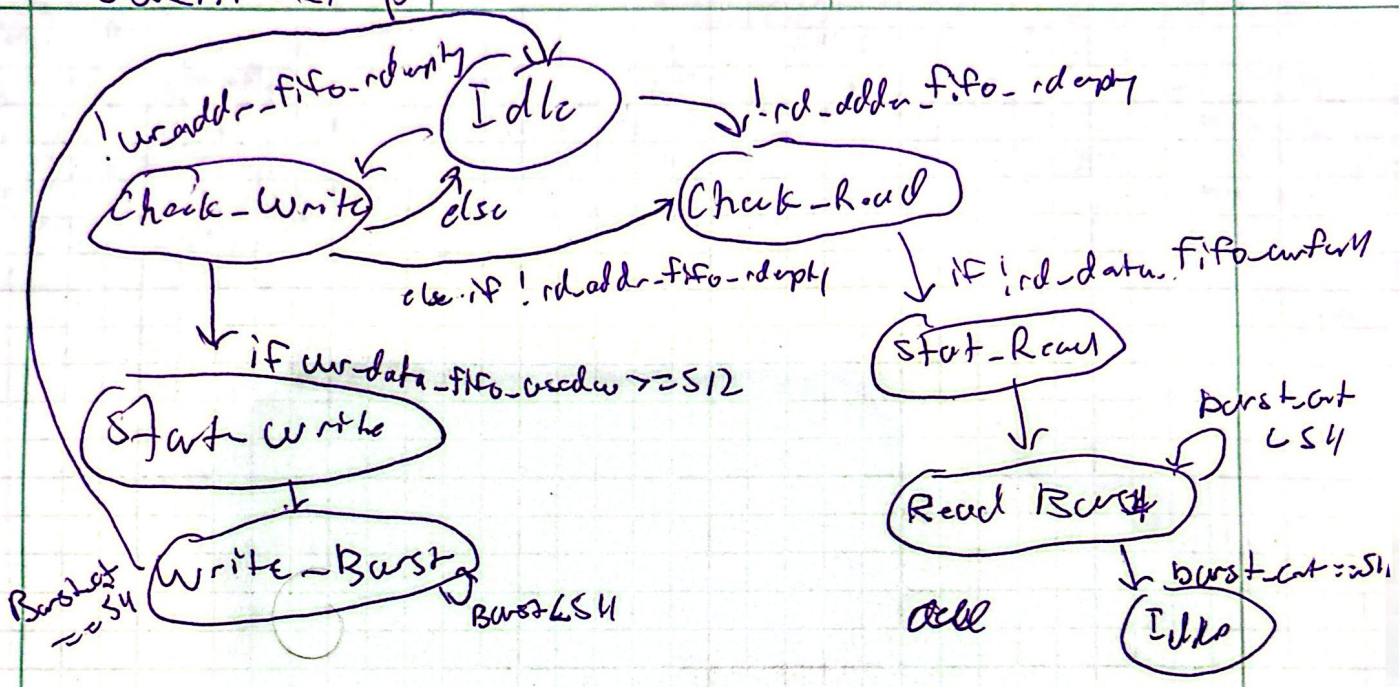
UART_mgr


Idle
rxnew_passdgc
else
Command_Type
if(C)
Control_Reg
data_wr_CH
if(R)
if(A)
if(W)
Read_Mem
Write_Mem
Wait_Mem
if sdram_rddata_valid
Prep_tx_mem
tx_mem_byte
ctr<7
Wait_tx_mem
if_tx_done&ctr==7
Mem_Incr_Addr
Address_Mem
Get_mem_addr
ctr<7
ctr==25
Idle
Get_Write_Data_Mem
ctr<7
ctr==27
Write_mem_end
else
if ctr<7
CR_Rdr_Wr
if R
W
Prep_tx_CR
Tx_CR_Byte
Wait_tx_CR
Rx_CR_Byte
ctr<7
ctr==7
ctr==2
Idle
Command_Type
W
Bulk Wr
Bulk_Get Addr
ctr<5
ctr==5
Bulk Get Data
ctr<4095
ctr==4095

SDRAM_Ctrl_v3



SDRAM_mgr


Idle
!wr_addr_fifo_rdempty
!rd_addr_fifo_rdempty
Check_Write
else
Check_Read
else if !rd_addr_fifo_rdempty
if !rd_data_fifo_wrfull
if wr_data_fifo_usedw >=512
Start_Read
Start_write
Burst_cnt <54
Read Burst
Burst_cnt ==54
Write_Burst
Burst<54
Idle